FINAL INTERNSHIP DEMO

YourTube Live Demo Script

A code-grounded walkthrough of the working product: accounts, playback, libraries, comments, plans, uploads and real-time watch parties.

Target 4:50 | Hard cap 5:00 | 7 feature blocks | 30s security | ~660 spoken words

Before you hit record

Run the whole sequence once. Reset any state you consume.

- [ ] Backend on **:5000** and frontend on **:3000**; both healthy.
- [ ] Main Chrome signed in as the demo user; second test user signed in in a separate window.
- [ ] Pre-open an incognito window at the OTP dialog, plus the admin moderation queue.
- [ ] Seed one normal video, one premium video, one non-English comment and one flagged comment.
- [ ] Prepare a small MP4 under 100 MB, a test channel, and a reset download/watch allowance.
- [ ] Confirm whether checkout is mock or Razorpay test mode, and which button appears.
- [ ] Grant camera/microphone permission in both party windows; close other apps using them.
- [ ] Hide bookmarks, terminal output, email inbox, API keys, **.env** and dashboard secrets.
- [ ] Use 1080p, 100% browser zoom, a clean desktop and a slow, deliberate cursor.
- [ ] Rehearse with a timer; aim to finish at **4:40-4:50**, never past 5:00.

> Only speak the **SAY** text. The grey boxes are stage directions and must not be read aloud.

Intro

0:00-0:15

15 SEC

ON SCREEN

- Start on the YourTube home page, signed in.
- Slowly scroll the video grid once; keep the cursor still while speaking.

SAY

Hi, I'm [Your Name]. This is YourTube, a full-stack video platform built with Next.js, React, Express, MongoDB, Socket.IO and WebRTC. I'll show the working product: accounts, video, plans, comments, uploads and watch parties.

Account security and personalization

0:15-0:45

30 SEC

ON SCREEN

- Open the profile menu, then toggle the theme once and return to the same page.
- Switch briefly to the prepared incognito window showing the "Verify it's you" OTP dialog, then return.

SAY

Sign-in uses Google through Firebase. The backend verifies Firebase's signed ID token before issuing a seven-day JWT. On a new browser, device ID, user-agent hash and coarse city or state are checked; unknown combinations require a six-digit email OTP. Verified devices are remembered. Theme is account-level: auto light from ten to noon IST, otherwise dark, and manual choices persist.

Discovery, streaming and plan gates

0:45-1:20

35 SEC

ON SCREEN

- Use the header search, open a normal video, then seek once in the player.
- Hover the player controls; show the free-plan ad-free notice.
- Open the prepared premium video and show the locked upgrade screen.

SAY

Home loads every video from MongoDB in a responsive grid; header search matches title or channel. Opening a video uses an authenticated streaming endpoint with HTTP range support, so playback can seek without downloading the whole file. Free plans get sixty minutes of watch time, tracked by backend heartbeats. Premium videos show a locked screen unless the plan includes premium access, and free users see an upgrade prompt instead of an ad-free badge.

Personal library and engagement

1:20-1:50

30 SEC

ON SCREEN

- Like the video, save it to Watch Later, then click Download once and cancel the browser save dialog.
- Rapid cuts through Liked, History, Watch Later and Downloads.
- End on Subscriptions to show remaining watch time and downloads.

SAY

Likes and dislikes are paired toggles that update the stored counters; Watch Later and History are per-user collections. History records the first view but still increments the video's view count on each watch. Downloads shows the current plan's daily quota and download history, while Subscriptions combines remaining watch time, remaining downloads and plan benefits. Saved items can be removed or reopened.

Comments, translation and moderation

1:50-2:20 · 30 SEC

ON SCREEN

- Post a short prepared comment; react to it; click Translate on the non-English comment.
- Open the Report dialog, choose a reason, then close it without submitting.
- Cut to the admin moderation queue and show Approve and Remove.

SAY

Comments are more than a text box. Each post is checked for length, abuse, spam and duplicates before storage. The server detects language and returns aggregate reactions plus the viewer's own state. Users can edit or delete their own comments, react, report with a reason, and translate any comment. Location is opt-in and coarse: state and country, never GPS. Three reports flag a comment for admin review; nothing is deleted automatically.

Plans, checkout and billing

2:20-2:50

30 SEC

ON SCREEN

- Open Upgrade and compare the four plan cards.
- Start Bronze checkout; use the labelled demo checkout or Razorpay test checkout.
- After success, show the updated plan and payment history.

SAY

The upgrade page pulls plans from the backend and compares Free, Bronze, Silver and Gold by price, daily downloads, watch minutes, premium access and ad-free entitlement. Checkout uses Razorpay when keys exist; otherwise a labelled mock checkout exercises the same flow. The backend verifies Razorpay's HMAC signature, upgrades the user for one month, emails an invoice, and records the payment. Verification is idempotent, so repeated callbacks cannot double-upgrade.

Channel and creator upload

2:50-3:20

30 SEC

ON SCREEN

- Open the profile menu to point out "Create Channel", then open the prepared channel.
- Select the small MP4, add a title, toggle Premium, click Upload, show progress and success.

SAY

Users can create a channel from the profile menu. The page shows the owner's description and uploads. Owners also get an upload panel: title, a one-hundred-megabyte MP4 limit, and a premium-only switch. Multer stores the file, the backend saves metadata and marks the video premium when requested, and it appears on the channel. The uploader is intentionally limited to MP4, matching the server's file filter.

Real-time watch party

3:20-4:00

40 SEC

ON SCREEN

- Two windows side by side. Host clicks Watch party; guest opens the copied invite link.
- Host plays and pauses; guest confirms sync. Guest sends one chat message.
- Host joins the video call; briefly toggle mute and camera, then share screen for two seconds.
- Host starts and stops a two-second local recording, then ends the party.

SAY

From any video, Create Party returns a six-character code and shareable link. Everyone joins through authenticated Socket.IO; rooms are capped at eight people, and empty parties expire after ten minutes. Play, pause and seek are broadcast, and late joiners use server time to sync. Chat is rate-limited to five messages per five seconds. The call uses WebRTC mesh, with Socket.IO relaying offers, answers and ICE candidates. The host can share their screen, record a local WebM and end the party for everyone.

Architecture and security

4:00-4:35

35 SEC

ON SCREEN

- Cut to VS Code with four files prepared: auth middleware, OTP utility, comment validation and party store.
- Highlight the JWT verify, HMAC hash, rate-limit and TTL lines only. Do not open
  .ENV
  .

SAY

Architecture-wise, a Next.js frontend talks to Express and MongoDB, with Socket.IO on the same HTTP server. Identity is layered: Firebase proves the Google account, the backend issues its own JWT, and every protected route re-verifies it. OTP codes are random, HMAC-hashed in memory, single-use, expire in five minutes, and are capped per email and IP. Auth and comment routes are rate-limited. WebRTC media is peer-to-peer; the server never carries audio or video. Party state is intentionally in-memory, limiting it to one Node instance.

Close

4:35-4:50

15 SEC

ON SCREEN

- Return to the YourTube home page; stop moving the cursor.

SAY

That's YourTube: authenticated streaming, plan-gated features, moderated comments, uploads, payments and real-time watch parties. The next step would be Redis for party state and object storage plus a CDN for media. Thanks for watching.

Do not claim or demo

These are real limitations in the current build. Avoid accidental overclaiming.

- The category strip changes its selected style only; it does not filter the home grid.
- The Subscribe button on a channel is client-side only; it is not persisted.
- There is no ad-delivery system. Plans control the **adFree** entitlement and UI badge.
- The upload form lists several formats, but the server accepts MP4 only. Demo an MP4.
- The 100 MB upload check is client-side; the server has no matching size cap.
- History and download-list reads are not auth-gated in the current code; avoid claiming every read is protected.
- Watch-party rooms live in memory on one Node process; they do not survive a restart or scale across instances.
- If Razorpay keys are absent and mock mode is off, checkout returns a configuration error. Verify the mode before recording.
- Never show **.env**, terminal logs, API keys, Razorpay secrets or the email inbox on camera.